Technical Requirements: Table Tennis League Management

Revision 0.2 - 17 October 2026. Based on the source document dated 17 October 2026, attributed to @kelliosli. This revision records the user's confirmed Next.js/Supabase, single-league and multiple-staff-role choices. It is a requirements baseline, not a claim that the application implements the league. Preserve the original 16-page PDF unchanged.

Revision summary

Retain FR-1 through FR-18 and the league business rules. Replace Django/Channels/Redis/Celery/VPS prescriptions with Next.js, Supabase and Netlify-compatible hosting. Staff commands travel over HTTPS; committed public notifications use Supabase Realtime; private staff status uses authorized polling. Replace the Argon2 prescription with verified Supabase-managed password security. Keep HTTP-only staff sessions, privacy, transactional integrity, capacity, backup and recovery targets. Unsettled rules are listed explicitly in section 8 and docs/DECISIONS.md.

1. Overview and scope

Run one youth table tennis league from season setup through teams/rosters, seeding, scheduling, live hall scoring, ratings and public results. Target release is 30 November 2026, with feature freeze on 23 November. The proposed schedule assumes two backend/domain engineers and one frontend engineer; re-estimate for different staffing.

Item	Requirement
Competition	Seasons, tours, stages and data-defined composite formats
Scale	24 clubs, 72 subteams across U13/U15/U17, 360+ players
Backend/frontend	Next.js App Router, React and TypeScript
Data/services	Supabase PostgreSQL, Auth, Storage and Realtime
Hosting	Netlify-compatible Next.js deployment plus separate managed Supabase staging/production
Match day	Approximately 1,000 viewers and 15 concurrent matches; 12 configurable tables by default
UI languages	Russian, Kazakh and English; Russian default/fallback
Exclusions	Player accounts, payments, native apps; CSV importer deferred, schema included

Age group and division are synonyms. A team is a club with three subteams, one per age group. A tour is called a tournament in the public interface. A team match is up to five singles games; a singles game is best of five sets. Staff are authenticated people; players are separate league records.

2. Domain rules

2.1 Competition and eligibility

A season has one or more tours configured before it starts; each tour has its own ordered stages. A stage uses round-robin groups, single elimination or a composite. Composite formats are ordered stage templates plus placement-to-seed mapping data, not hard-coded special cases. Generate groups using subteam ratings and preserve the seeding snapshot. Formats actually used this season, qualifiers and tie-breaks must be agreed before implementation.

Determine age eligibility using year of birth only, with explicit season reference year/cutoffs. Do not infer birthday-based eligibility. Players can drop out mid-event: preserve all results/history and reject new lineup selections for a dropped-out player. Active roster capacity is at most ten per subteam, without a roster gender quota. Playing-up and transfer rules remain decisions.

2.2 Team match and lineups

Slot	Singles game
1	First boy versus first boy
2	First girl versus first girl
3	Second boy versus second boy
4	Second girl versus second girl
5	Third boy versus third boy

The Trainer submits the lineup before the match; submission locks it. Each side's lineup is hidden from the opponent and public until both submit. Reveal both in one committed revision; networks may deliver that revision at different times.

The source requires at least three boys and two girls while also specifying an automatic singles loss for an unfilled slot. The valid partial-lineup and both-empty cases must be resolved explicitly. A no-show team loses the match. Only Main Judge or Organizer may replace a player after lock, including replacing a player who drops out; record old/new player, reason and actor. Mid-game score and Elo attribution need a signed-off rule.

The team match ends when one side reaches three singles wins. Cancel the remaining games automatically and exactly once; do not invent rating results for unplayed games.

2.3 Singles scoring

Singles are first to three sets, maximum five. A set reaches eleven points with a two-point margin; deuce can continue indefinitely. The Line Judge chooses the initial server. Serve rotates every two points and every point from 10:10. Change sides after sets. Derive the current server from point history and initial-server input, not a separately editable indicator. Cross-set/deciding-set details must be finalized before scoring implementation.

Record points individually. Undo point and undo set append events; they never delete history. Provide a timeout action for either player. Timeout limits/duration and reopening a completed result require section 8 decisions. Judges see player names, score, serve, connection/freshness state and large controls on landscape tablets. No optimistic score updates or offline scoring.

2.4 Elo

Each finished played singles game updates both players' ratings automatically in the result transaction. A player without an external rating starts at 1000. The proposed standard formula is EA = 1 / (1 + 10^((RB - RA)/400)); new RA = RA + K * (SA - EA), where SA is one for a win and zero for a loss. Confirm the 400 scale and numeric precision/rounding before implementing it.

Set K before a tournament; an authorized later K change affects only results not yet calculated. Store the K/settings version used in every history row; never silently recalculate past history using a new K. Automatic empty-slot losses do not change Elo. Completed-result undo must use the agreed compensating/revision model.

2.5 Subteam rating and seeding

Subteam rating is the sum of five selected eligible players' individual ratings. Proposed pre-match seed input: top five eligible players with three boys and two girls at a declared freeze time. Snapshot players, ratings and total so later changes cannot reshuffle a published draw. The proposal, tie handling and eligibility reference require sign-off.

2.6 Season ranking

Record each subteam's placement in each tour. Lower aggregated season score ranks higher; use head-to-head for ties. Proposed aggregation: sum of placements times configurable tour weights, default one. Confirm weighting, missing tours and multi-way/cyclic ties. Stale queued standings must not decide stage advancement.

3. Roles and functional requirements

Organizers create staff accounts, reset access and deactivate users. One person may hold several explicitly assigned staff roles. Permissions combine only the granted roles, remain resource-scoped and are rechecked on every private request/database command. An inactive account has no staff authority even with a still-valid token. Players have no accounts; spectators do not log in.

Actor	Authority	Restrictions
Organizer	Seasons, tours, stages, staff, settings, teams and all Main Judge operations	Audited, active staff identity; no metadata-derived authority
Main Judge	Run matches, start/start-all, manual table allocation, locked player replacement, manage all teams	No account/season creation or tournament-setting edits through this role
Trainer	Own club and assigned subteams: players, roster, lineup and upcoming matches	No other club; no locked lineup edits
Line Judge	Point/undo/timeout controls on the table assigned for that venue date	No other table or unrelated administration through this role
Public viewer	Published matches, timetable, draws, team lists, rosters and live scores	No writes or hidden lineups/private player fields

Trainer-to-subteam assignment is many-to-many within an authorized parent club. Explicitly bootstrap the first Organizer; a legacy organization owner is not automatically an Organizer. Scope access in RLS/RPCs independently of UI visibility.

ID	Functional requirement
FR-1	Create seasons, tours, stages and composite templates before season start; freeze configuration at start.
FR-2	Organizer creates staff, resets access and deactivates accounts; no public signup or player accounts.
FR-3	Configure versioned K-factor, table count (default 12), player rest time and finals priority.
FR-4	Trainers create/edit eligible players and rosters, max ten active players per subteam, no roster gender quota; validate birth-year assignment.
FR-5	Record dropout date/reason/actor while retaining every historical result.
FR-6	First/personal trainer reference is a linked staff trainer or plain first/last name, never both.
FR-7	Seed groups/brackets from frozen subteam ratings; Organizer may adjust before publication.
FR-8	Auto-allocate tables honoring rest, no simultaneous player games and finals priority; Main Judge overrides with audit.
FR-9	Validate, submit and lock lineups; hide them until both submit and reveal atomically.
FR-10	Match lifecycle scheduled, called, in_progress, finished; called highlights hall/public screens, human makes spoken announcement.
FR-11	Main Judge starts one match or all ready matches currently assigned to tables.
FR-12	Landscape judge interface shows names/serve/live score and point, undo-point, undo-set and timeout controls for either player.
FR-13	Persist every scoring command and stream committed updates; full audit records actor/time/before/after.
FR-14	Finish at three singles wins and cancel remaining games; apply agreed empty-slot/no-show rules.
FR-15	Recalculate Elo synchronously per finished played singles game and preserve per-game rating history.
FR-16	Compute tour placements, ascending season score/ranking and agreed head-to-head tie-breaks automatically.
FR-17	Public matches, timetable, draws, teams/rosters, player/ranking pages and live scoreboard in RU/KK/EN.
FR-18	Later CSV import of initial ratings; provide external-rating IDs and import/history provenance schema now.

3.1 Frontend structure

Canonical public routes sit below /ru, /kk or /en; / redirects to /ru. Include home/live header and league/individual ranking summary, /ranking, /player/{id}, /seasons, /season/{id}, /tournament/{tourId}, tour matches/match detail/draws, /live/{matchId} and /scoreboard. Player pages show best wins, rating graph and tournament match history. The source's ttfr.ru example is a visual reference only: do not copy its public date-of-birth columns into this minors' league.

Staff use /organizer for seasons/settings/staff, /hall, /team, /team/{teamId}, /subteam/{subteamId}, /subteam/{subteamId}/tour/{tourId}, match lineup/management and /linear-judge. Team and subteam IDs must be explicit; these are not organization tenancy routes. Auth callback paths remain fixed and safe. Exact route/access mapping is in docs/ROUTES.md.

4. Architecture and stack

Layer	Revised requirement
Application	Next.js App Router and TypeScript, server components by default, Server Actions and Route Handlers, Zod validation
UI	Existing Tailwind/shadcn components and tokens; responsive light/dark themes, translated labels and accessible controls
Auth	Supabase Auth; active staff and assigned roles stored in database, not user metadata
Database	Supabase PostgreSQL; constrained functions/RLS, row locks, immutable histories and atomic mutations
Real time	Supabase Realtime for committed public-safe notifications; snapshots remain authoritative
Staff commands	Same-origin HTTPS to Next.js; one user-authorized PostgreSQL RPC per scoring command
Private staff status	Authorized no-store polling/snapshots; never expose staff JWTs to add private browser subscriptions
Delayed work	Durable SQL jobs/outbox plus supported scheduling for ranking refresh, thumbnails and later import; retries/deduplication
Files	Private Supabase Storage; explicit consent and approved photo delivery/revocation; separate object backups
Deployment	Netlify-compatible Next.js plus separate staging/production Supabase projects; local Supabase for development
Observability	Error tracking, structured logs, transport/command/job/database metrics and event-day alerts
Recovery	Managed PITR capability plus encrypted off-site exports, object backups and measured restore rehearsal

Keep score/result/Elo/status/audit changes synchronous in one database transaction; delayed jobs may not award points or mutate official results independently. A pure TypeScript replay/scoring module must match authoritative SQL transitions through shared vectors and generated-command parity tests.

Business queries carry the verified caller's JWT. Only isolated server-only Organizer-authorized Auth provisioning/reset/deactivation may use an admin secret. Track provisioning across Auth/database as retryable pending/active/failed operations; do not claim cross-service atomicity. Next.js proxy refresh is not

the only permission gate. Every action/handler/RPC reauthorizes.

Secure HTTP-only staff cookies are server-refreshed. Anonymous public Realtime uses a separate client without staff Auth persistence. Prove refresh and recovery in an integration spike; this differs from default browser-readable Supabase sessions. The source Argon2 prescription is revised to Supabase-managed password security, whose configured guarantees must be verified. Never claim an unverified password hash algorithm.

5. Data model and invariants

Use UUIDs, created/updated timestamps, foreign keys and reviewed migrations. Preserve existing profiles, organizations, membership and audit history during transition; new league authority is separate.

Record family	Required fields/relationships and invariants
staff_accounts / staff_roles	Auth/profile ID, active/provisioning state, explicit coexisting roles; unique staff/role assignment, no self-grant
trainer_profiles / assignments	Trainer identity, private contact, team/subteam scope; no cross-club assignment
teams / subteams / season entries	Club name/region; unique team/age group; season participation and roster history
players / memberships	Names, gender, hand/style, DOB, sports level, rating/status, external ID; eligible active membership cap ten under races
player_trainer_refs	First/personal kind; linked trainer or free first/last name
player_status_events	Active/dropped-out status, effective date, reason and actor; preserve history
photo consent	Asset/evidence, consent status/version, actor/time and revocation handling
seasons / tours	Dates, locked configuration, tour ordinal and placement weight
tournament_settings	K/table count/rest/finals/version/effective time/actor; immutable versions
stage_templates / stages / groups	Ordered format data, qualifier/seeding maps and validated configuration
stage_entries	Subteam, seed, frozen eligible players/ratings and draw revision
standings / placements	Stage/group game/set/point differences and place, tour placement, season score/rank/tie evidence, source revision
team_matches	Participants, stage/group/round, status, table/times, winner/result type and version
lineups / lineup_slots	Unique match/subteam, submit/lock actor/time, slots 1..5, gender constraints and hidden/revealed revision
lineup_replacements	Old/new player, slot, reason, privileged actor/time; approved mid-game semantics
singles_games / game_sets	Match/slot/players/status/winner, set number/points/initial server/result; 11-point win-by-two checks
point_events / receipts	Ordered append-only commands/results, UUID idempotency, normalized payload, actor, before/after and versions

Record family	Required fields/relationships and invariants
rating_history / rating_imports	Initial/import/game provenance, before/after, K/settings/game; approved compensating correction history
hall_tables / table_assignments	Default twelve active tables, planned times, auto/manual actor, conflict/rest checks
line_judge_assignments	Judge/table/venue day, unique judge/day; enforce table/day for every score command
audit_logs	Actor/effective role/action/entity/before/after/time/trusted request context; append-only; retain legacy context
public projections / jobs	Only approved published columns; durable job ID/deduplication/revision/retry state

Database constraints and RPCs reject locked-lineup bypass, invalid slots/results, unauthorized actors, duplicate commands with changed payloads and simultaneous player/table use. A played team result needs three singles wins; forfeit/no-show exceptions must be explicit. Direct application writes cannot bypass audit or scoring invariants. Accounts/players soft-deactivate; preserve history and prevent destructive cascades. Index event sequence, match status/schedule/table, player membership/status and assignment lookups.

6. Real-time scoring and recovery

6.1 One command

The judge sends a UUID command and expected match version over HTTPS. Next.js validates shape/session. The RPC checks active role, table/day assignment and match/game state; returns an existing matching receipt before rejecting a stale version. A reused ID with changed payload is a conflict.

Lock team match/game/set and player rating rows in a stable order. Recompute the permitted scoring transition in PostgreSQL. Append command/event, update score/game/team, synchronous Elo/history, required advancement/public projection and audit together. Roll back all changes if any step fails. Only committed notifications reach viewers. Acknowledgement includes committed version/snapshot. The client never awards a point before acknowledgement.

6.2 Streams and access

Logical stream	Revised transport and safe content
Match detail	Public Supabase notification plus authoritative published score snapshot
League live board	Compact public called/live projection; no hidden lineup fields
Judge table	Authenticated HTTPS command response and private no-store status snapshot
Hall control	Organizer/Main Judge-authorized polling/snapshots for allocation and submission flags
Trainer team	Own-team authorized snapshots for upcoming matches, deadlines and reveal state

Every change increases the match version. Subscribe then fetch; resync on gaps, reconnect and refocus, and ignore stale in-flight reads. Public projections must exclude private base tables. Both lineup sides become visible in one committed database revision.

6.3 Connection behavior

Public Realtime reconnects with backoff/jitter, resubscribes and refetches the snapshot. Public fallback polls a cached safe endpoint every five seconds. Configure/test transport heartbeats rather than assuming the original Channels-specific twenty-second timer exists.

The judge connection indicator reflects HTTPS command/snapshot reachability and version freshness. Disable scoring if disconnected, stale or a command is unresolved. Reconcile an unacknowledged command using its original UUID before accepting another point. Private polling is bounded, stops when hidden and is never shared between sessions. Apply origin/CSRF protection and rate limits to mutation endpoints.

6.4 Acceptance latency targets

Metric	p95 target	p99 limit
Command received to transaction committed	100 ms	250 ms
Tablet tap to acknowledgement	300 ms	600 ms
Tablet tap to viewer-visible update	1 s	2 s
Live connect and first snapshot	1 s	3 s
Reconnect and resync	3 s	8 s
Cached public REST read	150 ms	400 ms

These source targets need measurement and final sign-off, not an assumption that a provider tier meets them. Include serverless/managed service latency and venue network conditions in the test. Benchmark fan-out early; change notification strategy if necessary while retaining PostgreSQL authority.

7. Non-functional requirements and acceptance

Area	Requirement
Capacity	Normal target 1,000 viewers/15 matches; acceptance 2,000 viewers/30 matches for 30 minutes with the latency targets met
Availability	99.9% scoring/public availability on event days; monitor and alert the team
Backups	Encrypted off-site recovery copies, 14 daily and 8 weekly; managed PITR capability for five-minute RPO
Restore	Measured thirty-minute RTO; restore database/Auth/grants/RLS and Storage objects in isolation before launch and on schedule
Languages	RU/KK/EN, URL locale, Russian default/fallback, localized dates/numbers and Kazakh Cyrillic support
Security	HTTPS, secure HTTP-only staff cookies, verified sessions, CSRF/origin controls, provider login limits, role/resource checks and append-only audit
Personal data	Public name/team/age group/rating/consented photo only; private DOB, contacts and consent evidence
Audit	Every score/undo, lineup submit/replacement, table override and settings change records actor/time/before/after
Integrity	Score/game/result/Elo/status changes in one transaction; preserve historic K and immutable correction events
Devices	Landscape judge tablets, mobile public pages, large hall scoreboard; installable web shell without offline scoring
Observability	Errors, structured logs, command/commit/transport latency, connections, jobs, database/storage capacity and tested event-day alerts
Delivery	CI each push, reviewed migrations, repeatable deployment, staging matching production and rollback/cutover rehearsal

Unit tests cover set/team results, serve rotation, Elo, seed/advancement and tie-breaks. Real integration tests cover permission denials, lineup secrecy/replacements, concurrent scoring and idempotency. Test recovery, copy/locales, tablet touch controls and privacy separately from UI visibility.

Go-live requires a conflict-free 72-subteam draw/schedule; thirty simulated matches through target tablets for thirty minutes with 2,000 viewers; latency and reconnect targets met; and a successful isolated restore. Configure sufficient simulated tables for load testing rather than violating the default twelve-table resource constraints. Verify service quotas/costs/region early. Database-only backups exclude Storage bytes; restore and reconcile photo/consent state separately.

8. Milestones, risks and open decisions

Milestone	Proposed dates
Rebaseline requirements and decisions	17-20 October
Staff foundation	20-25 October
Seasons, clubs and players	23 October-1 November
Draws and lineups	27 October-8 November
Scoring and Elo	2-12 November
Hall operations	6-15 November
Public pages, localization and live delivery	9-20 November
Integration/load/recovery readiness	16-22 November
Feature freeze	23 November
Rehearsal, fixes and release target	23-30 November

Overlaps assume the stated three-person team and stable contracts. Freeze UI strings by 9 November. Dates are targets, not evidence of completed features. Each coherent batch has exact files/migrations, tests/build checks, regression/permission review, evidence and a focused Git commit.

Risk	Mitigation
Six-week deadline	Resolve rules early; defer only approved optional work such as CSV execution or hall-screen polish, not integrity/privacy
Open-ended formats	Data templates for the actual season; reject unsupported maps explicitly
Formula/correction uncertainty	Confirm Elo, season aggregation and completed-result undo before dependent migrations
Venue network/tablet failure	Test venue/backup uplink and approved Main Judge fallback; disable stale/disconnected commands
Provider fan-out/latency/recovery limits	Early production-like load and restore experiments; do not assume default service tiers meet targets
Translation delays	Early externalized strings, Russian fallback, review all required locales
Minors' data	Confirm hosting/consent process with appropriate reviewers; enforce public-safe fields and revocation

Open decisions and dependencies

Resolved: Next.js/Supabase, one league, coexisting staff roles and no player accounts. Remaining decisions: exact age cutoffs/playing up/roster transfers; partial and both-empty lineups; this season's groups/qualifiers/brackets/tie-breaks; seed freeze inputs; Elo rounding/K scope and weighted/missing-tour scores; undo cascades after completion; mid-game replacement score/Elo; first serve across sets/timeouts; judge fallback; rest/finals/manual override rules; timezone/devices/network; hosting region, consent process, provider recovery budget and alert recipients. See docs/DECISIONS.md for dependency IDs. Unsettled rules are not silently replaced by invented defaults.

Source and implementation status

Original source: Copy_of_Technical_Requirements_Document_Table_Tennis_League_Management.pdf (16 pages, 17 October 2026, @kelliosli). The original remains unchanged. This revised document replaces technical prescriptions only where the user's confirmed choices require it; it preserves functional IDs and operational acceptance targets.

At this revision, the website remains the original Phase 1 foundation. Task A changes documentation and provides this reviewable requirements copy. Each subsequent milestone must record its actual verified implementation status in docs/IMPLEMENTATION_PLAN.md.